

Digital electronics

DFT

-DFT

-Formal verification

تعالى نبدا مع بعض فى الجزء الأول

DFT:

خلينا قبل منبدا ندخل في تفاصيل اكثر نعرف بس هو اي ال DFT ده

ال **DFT - Design for Test** فكرته إن وانت بتعمل ال design نفسه، تضيف جواه شوية حاجات أو تعديلات تخلي عملية اختبار Chip بعد التصنيع سهلة ومضمونة. الفكرة إنك مش بتكتفي بال verification اللي بتعمله في ال design stage بال simulation ، لأن ده بياكدك إن ال design شغال logically بس. إنما بعد ما Chip تتصنع فعليًا، ممكن يظهر مشاكل في التصنيع نفسه تمنعها من إنها تشتغل. هنا بييجي دور ال DFT: بيخلي عندك إمكانية تعمل اختبارات Chip الحقيقية وتكشف العيوب دي.

الفرق بين Functional Test و Manufacturing Test

Functional Test (1)

ده بيكون في مرحلة التصميم، قبل التصنيع، باستخدام simulation tools زي ModelSim. الهدف إنك تتأكد إن التصميم بيأدي الوظيفة المطلوبة صح. حتى مع وجود routing و parasitics وال operating conditions، كل ده مجرد محاولة لتقريب الواقع. بس في الآخر، ده يفضل simulation مش hardware.

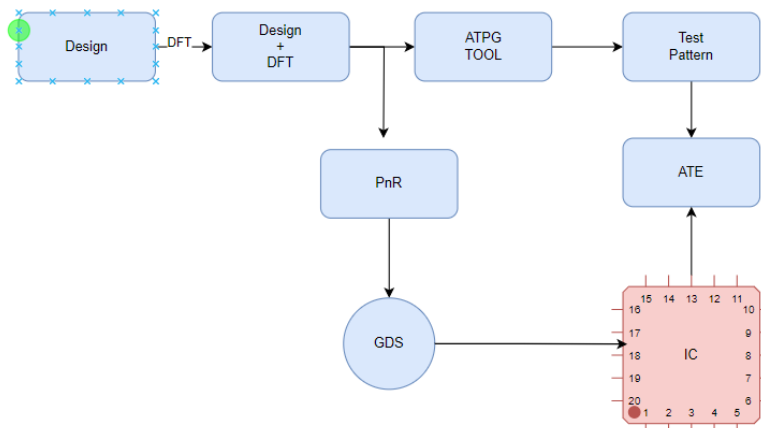
عشان كده في حاجات ممكن متبانש خالص غير في السيليكون الحقيقي، زي إن خطين بيقوا short أو يحصل violation في design rules. فال functional test مهم للتأكد من ال logic، بس مش كافي عشان يضمنك إن الشيب اللي طالعة من المصنع سليمة.

Manufacturing Test (2)

ده بيتم بعد ما الشيب تطلع من المصنع، والهدف هنا إنك تكتشف عيوب التصنيع زي cracks ، shorts، أو أي defect حصل أثناء التصنيع. الاختبار ده هو اللي هيحدد إذا كانت الشيب دي صالحة وتتبع للمستخدمين ولا لازم تترمي. وعشان العيوب دي بتكلف فلوس ويتخسر شركات كثير، لازم نكون مجهزين من الأول بحاجات تساعدنا نعمل الاختبارات دي، واللي هو دور ال DFT. لأنه من غيره الاختبار هيبقى صعب جدًا ومكلف، ولو الشيب بايطة خلاص هنترمي وتعتبر خسارة.

طب الكلام ده بيحصل ازاى؟

باخد ال chip دي ادخلها على (ATE TOOL) دي توول بتعمل تيسر للتشيب عن طريق Test Pattern والي بيجيه من ال ATPG Tool وده بيبقى فايل فيه جواه كل ال patterns الي محتاجها عشان اعمل تيسر للتشيب زي مهو واضح في الصورة



Defects (1)

ال Defect ده basically بيكون مشكلة بتحصل في مرحلة ال fabrication أو ال layout. يعني وانت بتصنّع الشيب ممكن يحصل حاجات زي:

- يحصل short بين خطين مالمش دعوة ببعض.
- أو open circuit يعني السلك اتقطع.
- أو يحصل extra metal يعمل bridging.
- أو حتى dust يدخل على ال wafer ويؤّظ الدنيا.

Faults (2)

ال Fault بقى هو ال translation بتاع ال defect ده في circuit level يعني لو حصل short في layout ده defect لما ده بيان في الدائرة إن ال output مثلاً stuck على "1" أو "0" → هنا نسميه stuck-at fault

Error (3)

ال Error هو النتيجة الغلط اللي بتظهر في ال output النهائي. مش كل fault بيعمل error ، لكن لو fault فعلياً أثر على ال function بتاعة السيكرت وخلي النتائج تطلع غلط → يبقى ده error

مثال: لو عندك 01 = input المفروض يطلع 1 = output ، لكن بسبب fault طلع 0 → هنا ده اسمه error

كل Error سببه Fault ، وكل Fault سببه Defect ، لكن مش كل Defect يعمل Fault ، ومش كل Fault يطلع Error.

ال Fault Models بتتقسم لحاجتين أساسيين Stuck-at و Transistor faults و Bridging faults

Stuck-at Faults (1)

النوع ده من الفولتس بيتعامل مع ال nodes على إنها ممكن تكون "محبوسة" على قيمة معينة. أول حالة اسمها Stuck-at-Normal ودي معناها إن ال node شغالة عادي جداً، أي قيمة بتدخل سواء 0 أو 1 بتظهر مطبوعة من غير أي مشكلة. بعد كده Stuck-at-0 ، ودي معناها إن ال node دايمًا شايلة 0 حتى لو دخلت 1 ، وكأنها متوصلة على طول بالـ GND. وأخيرًا Stuck-at-1 ، وده العكس بالضبط، إن ال node بتبقى دايمًا شايلة 1 حتى لو دخلت 0 ، وده كأنه short على VDD

Transistor Faults (2)

في النوع ده التركيز بيكون على الترانزستور نفسه مش ال node. أول نوع اسمه Stuck-open ، يعني الترانزستور مش بيشغل خالص وكأنه open circuit ، والسبب ممكن يكون threshold voltage عالي بسبب مشاكل في ال doping أو أي defect تاني يمنع الترانزستور من التوصيل. النوع الثاني هو Stuck-on (short) ، وده بيحصل لما الترانزستور يفضل دايمًا on كأنه short circuit. الحالة دي أخطر لأنها بتعمل path ثابت من VDD لـ GND وبالتالي يظهر static current عالي جدًا ، وكمان ال output ممكن يبان كأنه voltage divider بين VDD و GND

ال fault models دي مش بتوضح السبب الحقيقي للـ defect ، لكن بتمثل تأثيره على مستوى ال circuit

يعني مثلاً لو ترانزستور دايمًا on ، الموديل يقول ده stuck-open لكن مش هيقولك هل السبب doping عالي ولا gate oxide مشكلة

Bridging faults (3)

ده بيبقى عندي Nodes بقوا Shorted على بعض وده بيحصل برده اثناء التصنيع

انا دلوقتى هدفى الاساسى انى اعرف اكتشف ال faults دى وانا بعمل testing لل chip بعد اما تيجى ... فهل هشتغل على كل ال faults دى؟؟ لا احنا هنركز على Stuck at faults لان دول قادرين انهم يضمنوا بنسبة كبيرة اوى ان ال chip تمام لو انا اتأكدت ان مفيش اى faults stuck at 0 or stuck at 1 ومنهم هضمن ان باقى ال faults بنسبة كبيرة مش موجودة بس كده محتاجين نعمل testing لكل node فى ال chip فهل هقدر اعمل كده؟ اكيد هقدر مدام بندرسها يبقى قدروا يعملوها بس خلىنا الاول نشوف ال gate الواحدة ازاى هعملها testing عند كل node فيها عشان نفهم ال criteria الى ماشين عليها ثم نشوف بنطبقها ازاى على الواقع ... فناخد OR Gate كمثال نطبق ونفهم عليها:

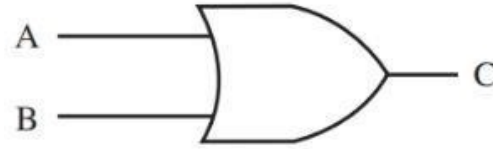


Figure (6) [OR Gate]

Notes:

- $C(A/0)$: output C when input A stuck at zero
- All red outputs shows the errors occurs due to stuck at faults

A	B	C	C (C/0)	C (C/1)	C (A/0)	C (A/1)	C (B/0)	C (B/1)
0	0	0	0	1	0	1	0	1
0	1	1	0	1	1	1	0	1
1	0	1	0	1	0	1	1	1
1	1	1	0	1	1	1	1	1

كل ال red outputs دى هى ال errors الى نتجت عن faults فى ال chip فعشان نفهم ان حصل fault محتاجين نشوف ال errors دى قدامنا عند ال output فهنا لاحظ ان كل error منهم حصل عند fault معين قدرما نشوفه عن values معينة من inputs وده بنطلق عليه pattern input يعنى قيمة ال inputs الى لو دخلناها هيقدر يظهر fault معين عندى ويطالعلى wrong output ولو طلع ال output مذبذب فهناك انه ال fault ده مش عندى فمثلا عند A stuck at 0 هلاحظ ان ال input pattern الوحيد الى بينلى حاجة زى دى هو 10 وهكذا فى كل ال faults الموجودة ومن هنا هلاحظ ان $\{00, 01, 10\}$ only input patterns هتكون كافية نكتشف كل ال faults وبالنسبة الى 11 فهو بيكشف C stuck at 0 ولكن فى two patterns تانيين معايا اصلا هيكشفوها فمش هحتاجه ويكفى ان كل fault يكون له pattern واحد بس ... ليه؟

لاينا بنحاول نقل ال input patterns بقدر الامكان لان فى الواقع انت عندك millions of gates على ال chip فأصلا حتى pattern لكل fault لكل node هيبقى كتير اوى فهناخد وقت كبير جدا عشان تعمل testing قد يصل الى سنين انا لو عندى combinational circuit ليها M inputs يبقى 2^M number of possible input patterns ولكن ال chip فيها combinational مع sequential يعنى عندى flipflops بفرض ان عددهم N يبقى بردوا 2^N Number of states فى حالة combinational بس

كان 2^M number of tests لكن بما ان عندى الاثنين يبقى **Number of Tests = $(2^M) * (2^N) = 2^{(N+M)}$ test** فلو عندى $M=40$ & $N=40$ ودى ارقام قليلة اصلا مقارنة بالواقع ولكن هتطلع عدد كبير جدا من tests هتاخد وقت كبير جدا

وده مش منطقي ففعليا على ال chip الواحدة انت مش بتطبق عليها كل ال input patterns الى هتكتشف عن ال faults كلها ... فوجدوا ان حسب figure 7 لو طبقت كل ال test patterns هتوصل ل full coverage بمعنى انك غطيت كل احتمالات الخطأ الممكنة وخلاص كده اكيد مفيش اخطاء ولكن زى ما قولنا عشان اعمل coverage لكل ال faults لكل ال nodes فى chip هتاخد وقت كبير جدا فعلا ممكن يوصل لسنين باستخدام ال tools عادى فهنا لاحظ ان جزء صغير من patterns كان كافى يوصلنا لنسبة coverage كويسة جدا مع بقاء error اكيد ولكن بسيط يعنى acceptable وال patterns دى بنختارها بحيث تكون كافيه وكويسة توصلنا للنسبة دى ومش احنا الى هنتاخر يعنى ال tools هتعملها وهنشوف ازاى بعد شوية

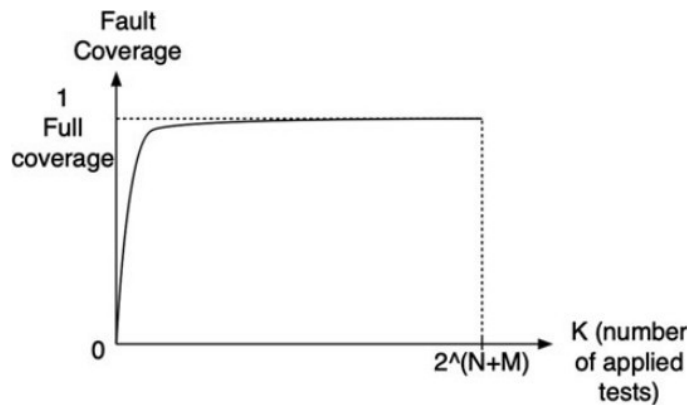


Figure (7) [fault coverage Vs applied tests number]

Note: We assume a single faulty model (one fault at a time) based on the fact that the probability of multiple faults occur is less than that of single fault occur

يعنى احنا بنعتبر ان fault واحد بس الى هيحصل ل node واحده عشان كده احنا كنا بنبص على node واحدة بس ونشوف تأثير ال fault عندها على ال output فى الجدول فوق ... وده فرض مقبول لأنها احتمالية حدوث Multiple faults بتكون قليلة

Controllability and Observability:

Controllability:

يعنى قدرتى انى اتحكم فى ال gate فاكيد عشان اتحكم فى حاجة هيبقى من خلال ال inputs وبالتالي مدى قدرتى انى اتحكم فى ال inputs وادخلها بالقيم الى عايزها واغيرها بسهولة بتحدد ال controllability لان لو عندى two gates واصلين ورا بعض فقدرتى على التحكم فى inputs الثانية اقل من الاولى لاني الاولى انا معايا ال inputs بشكل مباشر احطها واغيرها وقت ما انا عايز لكن الثانية لازم اشوف انا عايز عليها input بكام واشوف ازاي اقدر اطلعها من ال gate الاولى وعلى اساس ده احدد inputs الاولى الى سهل اتحكم فيها فمثلا فى figure 7 انا مش هقدر اتحكم فى x بشكل مباشر فمثلا لو عايزها 1 ببقى لازم ال A و B يكونوا 1 بردوا فالى بتحكم فيهم هنا مباشرة هما A & B & C فال control على AND اكبر من على OR

Observability:

يعنى القدرة انى اشوف ال output وبالتالي لو ال output ده قدامى على طول هيبقى افضل من ناحية ال observability لكن لو two gates واصلين ورا بعض فال output بتاع الثانية عشان اشوفه لازم اضبط باقى ال signals الى داخلة معاه عشان ي propagate زى ماهو ال output الاساسى الى قدامى واقدر اشوفه فمثلا فى figure 7 لو عايز اشوف x ببقى لازم احط C=0 لانها OR gate فلو x=1 تطلع على برا F=1 لو x=0 تطلع على برا F=0

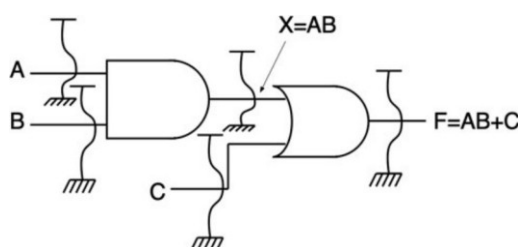


Figure (8)

وبكده نقدر نستنتج ان كل ما كانت ال gate اقرب لل inputs الاساسية (الى برا فى ايدي اتحكم فيها) هتكون افضل فى controllability ولكن اسوأ فى observability والعكس لو كانت ال gate اقرب لل outputs هتكون افضل فى observability ولكن اسوأ فى controllability

انا عشان اعمل اى testing لاي node فى ال chip وى ما شوفنا هبص على ال gate الخاصة بيها واحط inputs معينة واشوف ال output المقابل واقرنه مع ال golden output (الى هو ال output الصحيح الى انا مستنيها) فازاي وسط chip فيها gates كتير جدا جدا زى ما احنا عارفين هقدر اعمل testing واتحكم فى inputs & outputs لكل gate ... ممكن نعمل الطريقة الى استخدمناها فى figure 7 وافضل ارجع كده بال inputs لحد ما اشوف هحط ايه على ال inputs الاساسية بتاعتى وفى نفس الوقت اخلى ال output ي propagate بس هل منطقى حل زى ده فى وسط chip فيها millions of gates ؟ اكيد لا

فاحنا دلوقتى بنقول ياريت لو اقدر اتحكم فى inputs لكل gate عندى فى ال chip وياريت لو اقدر اشوف كل ال outputs الى عندى وقت ما احتاج على طول من غير ما اقعد احاول اضبط باقى ال signals ممكن نفكر نطلع pins من كل ال nodes دى برا على ال chip بس طبعا مش منطقة ببقى عندى pins كتيرة جدا بالشكل ده عدد ال pins اصلا بنبقى محكومين بيه وزى ما بنشوف فى اى chip بنشتريها انه بيكون عدد قليل جدا مقارنة بعدد ال nodes المهول فى ال chip فعملوا ايه؟

Scan Chain:

Scan flip-flop & scan chain:

ده ال technique الى هنستخدمه عشان ال testing ... فاحنا عارفين ان ال design بتاعنا بيكون عبارة عن combinational logics بين flipflops يعني عندى flipflop عند ال input ثم combinational logic ثم flip flop ثم combinational logic وهكذا لحد ما نوصل لآخر flipflop الواصل عليه ال output الأخير بتاعنا واحنا عايزين نعمل testing لل gates جوا ال combinational فأي node هحتاج اعملها test من الى شوفناهم اكيد فى المقابل عندى input لل combinational logic دى كلها هعوز أدخلها عليها وده بيتحدد ب tool مش احنا الى هنفكر فيه ... فما علينا المهم انى ببص دلوقتى على جزء combinational عندى لوحده (الى هو بين كل two flipflops) واقدر باستخدام tool هنشوفها اعراف ال input test patterns ال المناسبة له فهفضل المشكلة بردوا هوصل لل input ده بشكل مباشر ازاي وهشوف ال output الى فيه بشكل مباشر ازاي من غير ما اعقد اظبط كل ال combinational logics الى قبله وبعده لده؟

فكرة ال scan chain هي انى بحت بدل flipflop العادى **scan flipflop** وده عبارة عن Mux + flipflop زى ما واضح فى figure 9

9 فكدده انا اما هدخل لل flop ال date العادية D ويسلمها لل combinational الى بعده وهكذا العادى الى عارفينه او انى هدخله ال SI وده ال Scan input الى انا عايز اوصله لل combinational معينة جوا وبختار بينهم عن طريق Scan Enable SE بحيث لو 0 يبقى مش عايز ال Scan chain وعايز ال functional path العادى بتاعى ولو 1 يبقى هتشغل ال Scan chain وادخل SI طيب SI ده عندى ك input من برا وداخل على اول scan flipflop هوصله ازاي لل combinational logic الى انا عايزها؟

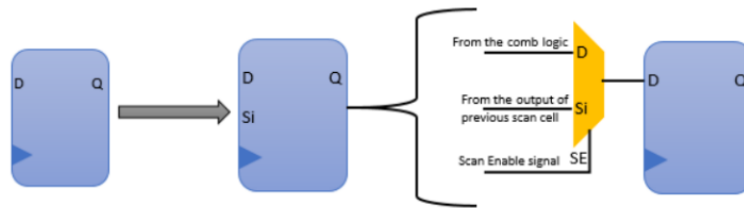


Figure (9) [Scan flipflop design]

نفس ال flop ده هيكون موجود مكان ال flos العادية (زى ما واضح من figure 10) وهيتوصل D بتاع كل واحد بناتج ال combinational logic الى قبله زى العادى ولكن SI هيوصل ب output ال scan flop الى قبله مباشرة قبل ما يدخل على ال combinational logic اصلا بحيث ان ال Scan chain دى هتكون زى shift register بتتقل ال SI من flop للتانى لحد ما اوصلها لل combinational logic الى عايزها وبعد ما يطلع ال output بتنشغل بردوا shift register وتطلع زى ما هو على برا وهنشوف بالظبط بتنشغل ازاي و Test ازاي فى figure 10 ال combinational logic الكبير بعد كل flop بيدخل Q output لل logic ثم يطلع ال logic نفسه output يدخل ك D input لل flop الى بعدى وهكذا وده العادى الى عارفينه فبتطلق عليه functional path لان ده ال path الى بيحقلل ال function الى انا عامل عشانها ال chip اصلا ... لكن ال path التانى (ال scan chain path) ده لل testing يعنى هستخدمه مرة واحدة اول ما ال chip تجلى وخلص

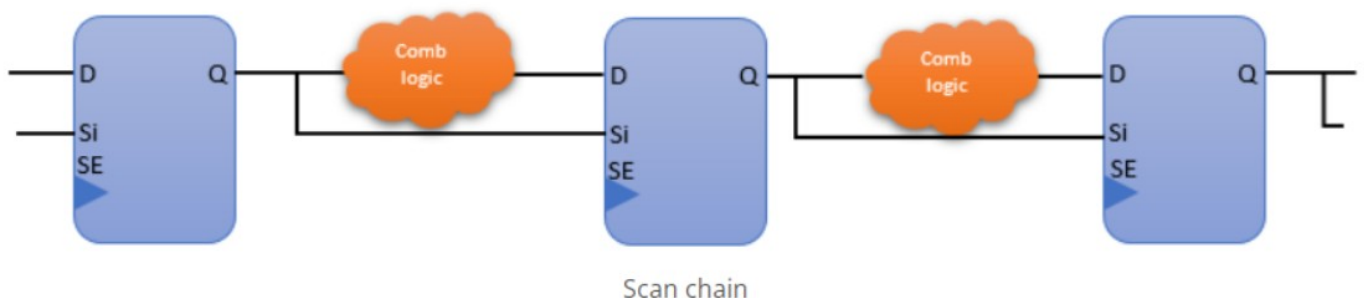


Figure (10) [Design with scan chain]

دلوقتی لو ان عایز عمل testing لتانی combinational هنا فھط ال test input الی متحد من ال tool ثم ھخلى SE تبقي 1 واستتی two clock cycles ھیحصل فیھم shift ال SI لحد ما توصل عند Q of the second flip flop بعد كده انا عایز عمل test ال combinational logic الی واقف عندها دلوقتی فاكید مش ھعمل shift ل SI لان انا عایز ادخلها واشوف ال output الناتج عنھا فھنا بنزل SE الی 0 تانی بھيٹ ان third flip flop یاخذ الناتج من combinational عادی یطلعھ ك output بعد كده لو لسھ فیھ باقی ال design وعندی combinational logics & flip flops تانی فیرجع ارفع تانی SE الی 1 بھيٹ انھ یاخذ ال output الی طلع ده وبعملھ Shift بردوا لحد ما یوصل ال output النھائی بتاعی الی بقدر اشوفھ من برا ال chip وبنطلق علیھ scan out SO

یعنی ال **Scan Chain Mechanism** ھیحصل انی اخلی 1 SE فی الاول بھيٹ اشغل ال test mode لعدد من clock cycles قد الی محتاجھ عشان اوصل SE ال combinational logic الی عایزھا ثم اخلی 0 SE بھيٹ اشغل ال functional mode عادی لمدة one clock cycle واشوف ال output من ال combinational logic (یعنی عمل capture ال output المطلوب ده من خلال ال flip flop الی داخل علیھ) ثم ارجع تانی 1 SE عشان ارجع ال test mode وابقی شغال shift register بھيٹ اطلع ال output ده برا عند SO فممکن نقسمھ الی ثلاث مراحل زی فی figure 11:

- Shift In (Scan In) – SE = 1
- Capture – SE = 0
- Shift Out (Scan Out) – SE = 1

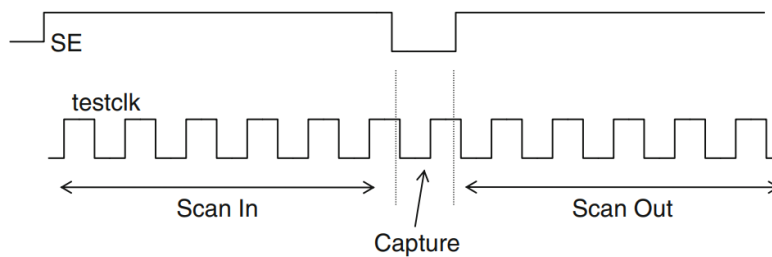


Figure (11) [Scan chain mechanism]

Test input pattern sequences:

الشكل فی figure 10 ده فی منه كثیر فی ال chip یعنی مش ثلاثة flip flops بس وكمان ممكن ال combinational logic الواحدة ببقی لیھا اكثر من input وكل واحد جای من flop مختلف زی فی figure 12 ... یعنی المختصر انھ بتحتاج تدخل pattern كامل ورا بعض سواء ھتعمل test لكذا combinational logic فی نفس الوقت او ان ال combinational الواحدة محتاجة inputs جاية من اكثر من flop عندی (فی ال chip مش بیکون كلھ واصل ورا بعض زی فی figure 10 لا بیکون فیھ كذا chain كده in parallel بردوا) ... احنا عندنا SI ھی bit one ... بس واحنا بنشوف ال mechanism كذا متابعین ال bit one ومتكلمناش عن ال bits التانية الی بتدخل علی كل flop كل cycle clock بتعدی لان مكش مهم بالنسبانا ای combinational logic تانية غیر الی بندرسھا لكن لو انا مهتم بانی اوصل كذا test input ل nodes مختلفة فی نفس الوقت بھيٹ اول ما انزل SE الی 0 لمدة one cycle كلھم یشتغلوا فی function mode ویطلعوا outputs ثم ارفع SE تانی ل 1 فتبدأ ال outputs دی یحصلھا shifting ویخرجوا ورا بعض علی SO

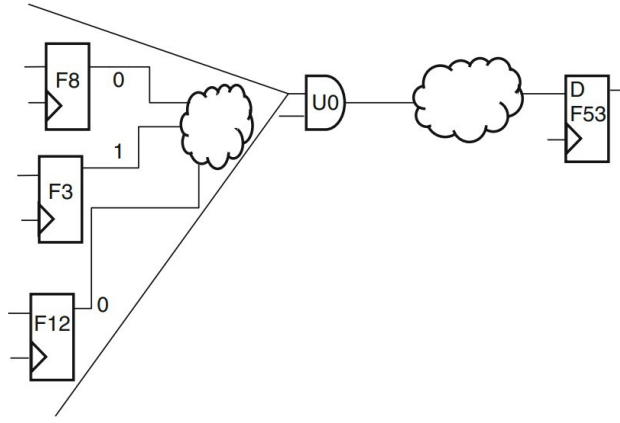


Figure (12) [Scan chain with input sequence]

ففى figure 12 نفترض ان رقم كل flip flop بيتمثل عدد ال cycles الى محتاجها عشان اطلع ال output بتاعه يساوى test input الى عايزه لل combinational logic وبفرض ان الارقام المحطوبة هى دى الى محتاجينها بيقى بعد twelve clk cycles عايز احط 0 وبعد eight clk cycles هحط 0 وبعد three clk cycles هحط 1 فهدخل على SI ال sequence المناسب وهو "0xxx0xxx1xx" من الشمال لليمين واحدة ورا الثانية و x يعنى don't care فالى حاجة عادى مكانها لانى مش مهتم بالى هيوصل لل nodes الى هنتف عندها القيم دى بعد twelve clk cycles وبالتالي بعد twelve clk cycles هكون وصلت للشكل فى figure 11 انزل 0 SE = لمدة one clk cycle ثم ارفعها تانى زى ما احنا عارفين

بنفس الفكرة لو عندى chain واحدة اقدر ادخل sequence بردوا بالطريقة دى بحيث مثلا لو F8 F3 F12 دول فى نفس ال chain وكل واحد بيطلع output يدخل على combinational يعنى محطوطين بنفس الشكل الى فى figure 10 ورا بعض فهدخل نفس ال pattern ده "0xxx0xxx1xx" ولما انزل SE الى 0 كل combinational logic من الثلاثة هيطلع الناتج بتاعه وابدأ اعمل shift يانى ارفع تانى SE الى 1 زى ما احنا عارفين فيطلع ال outputs عند SO بعض يعنى مثلا هيطلع بتاع F12 اول واحد ثم بعد three clks يطلع بتاع F8 ثم بعد four clks يطلع بتاع F3 وهكذا فكدنا انا قدرت اعمل testing لكذا node فى نفس الوقت

طبعا ال input sequences بتدخل ورا بعض يعنى خلصت sequence معين ب test بييه بعض ال faults فيعد ما اعمل capture ادخل sequence تانى لغيرهم ففى نفس الوقت هيكون second sequence shift مع output shift of the first

sequence فده بنسميه **overlapping Shift In and Shift Out** زى ما واضح من figure 13

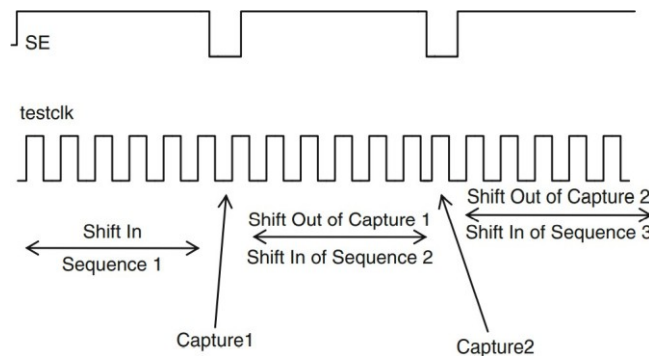


Figure (13) [Overlapping Shift In and Shift Out]

Disadvantages of DFT:

فكرة انى حطيت Scan flip flop بدل العادى دى اكيد ليها سلبيات وممكن قبل DFT يكون ال design تمام وعملتله & synthesis formal verification والدنيا تمام ولكن بعد DFT الاقى الدنيا باظت لان ال constraints الى انا حاططها ممكن متمشيش مع scan flip flop ... هو بيغير فى ايه بايه ال Scan flip flop ؟

- بالنسبة لل **Timing**: فوجود MUX زيادة هخلى فيه **higher setup requirements** لان ال path دلوقتى طول وبقي عندى mux زيادة والكلام ده سواء لل functional path بردوا لانها بيعدى على ال mux فمممكن يكون فيه paths كانت setup met وبعد DFT تبقى violated لان الوقت الى زاد عليا من ال mux فى ال path خد من ال slack الى كان المفروض يتساب لل setup وكمان هيبقى عندى **higher clk-to-q delay** لان دلوقتى فى load زاد على flip flop output لان من figure 10 هتلاحظ انه بقى متوصل ب nodes two واحدة واصلة بال combinational logic ودى العادية المتعارف عليها والثانية هى الى واصلة على SI بتاع ال flop الى بعده فدى اكيد ليها capacitance بردوا فتهزود ال load عن ال flop العادى وبالتالي clk-to-q delay هيزيد.

- بالنسبة لل **Area**: زادت اكيد وبنسبة كبيرة من 30% ل 50% من ال design وبالتالي ال silicon area الى هتاخذها زادت يعنى تكلفة اعلى ولكن مش عالية قصاد ال controllability & observability الكبار الى هاخداهم ولازم اعمل testing لل chip ودى اكثر طريقة تسهل عليا اعمله

Scan clk and Scan rst:

ممكن ان CLK & RESET الى يستخدمهم فى functional mode ميكونوش نفس بتنوع testmode فأنا بحط بردوا MUXs عند CLK & RESET قبل ما تدخل اصلا على Scan flipflop وبيكون ال selection بتاعهم مختلف عن SE ك signal ولكن بردوا بيبقى ب 1 وقت Scan ساعتها بيبكون input of mux هو scan rst or scan clk ووقت capture بيبكون 0 بحيث يدخل ال functional clk & functional rst واى كانت ال clk الى داخله هتبقى muxed مع ال scan clk يعنى حتى لو مش هى clk source مباشرة وكانت خارجة مثلا من divider or clk gating

Notes:

1- recommended number of flip flops in the chain is sometimes 100

ال chain لو طولت اوى بيبكون delay كبير اوى وبالتالي بت recommend عدد لل flops وليكن على حسب الى بنشتغل بيه هنا فهو 100 ولو احتاجت اكثر تعمل parallel chains ويكون عندك اكثر من chain فى ال design يعنى اكثر من scan path فكاكك قسمت ال system بالشكل الى فى figure 14 وكل واحد له SE SI SO .

2- لو مستخدم shift registers اصلا فى design (يعنى ال design بتاعك اصلا عبارة عن shift registers) فانت كده جاهز لل DFT ومش محتاج تحط scan flip flop ولكن ظبط ال enable بتاعهم حسب ال data الى هدخلها هتكون functional mode or testing mode

3- متحطش positive and negative triggered flip-flops مع بعض ... هو نوع واحد منهم استخدمه

4- مينفعش يكون عندى combinational feedback loop (يعنى output من combinational بيلف ويرجعها تانى) فلازم اكسر ال loop دى ب flip-flop بردوا واحاول اتجنب وجودها اصلا لانها بتعمل مشاكل فى timing analysis

5- بالنسبة لل clock gating cells انت فى ال synthesis بتخلى ال tool تجيب ال cell الى عندها أغلب الوقت ومش بتعتمد على ال clock gating block الى انت عامله فى RTL (لازم تعمله فى RTL عشان تقدر تشوف نتائج ال simulation بتاعك) فعامه هى عبارة عن الشكل فى figure 16 (من غير OR gate فى العادى) فممك أثناء ال testing يكون فى حاجة فى ال code الأساسى (الى شغال عادى بس انا مش باخد النتائج بتاعته وبعمل shifting لل values الى عايزها بس) ممكن الحاجة تيجى تكون بتقفل ال clock gating لفترة يعنى بتخلى $gate_enable = 0$ واكيد احنا مش عايزين فى وسط ال testing نقل ال cell وبالتالي بنحط مع ال $gate_enable$ فى OR logic واحدة من ال signals المسؤلة عن تحديد انا فى انى mode الى هما scan_enable or test mode وهنا بنقول اننا عملنا bypass لل gated clocks فى ال test mode

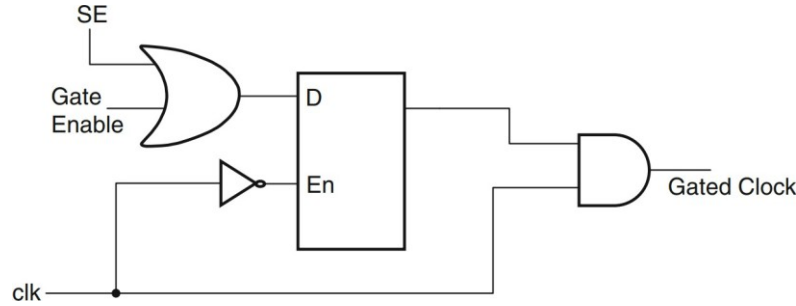


Figure (16) [Clock gating cell considering the test mode]

Inserting DFT in your Design:

كل ده بنتكلم عن DFT شكله ايه وفائدته ايه وتفصيل كثير لكن لسه مقولناش هحطه بقى ازاى فعلا جوا ال design ... اكيد هستخدم tool وبردوا كالعاده عمل script وهو يقوم بالمهمة دى ويبدل flip-flop ب scan flip-flop فأكد عندى constraints هحطها لل tool بردوا فخلينا نشوف ايه الى هعمله عشان اضيف بالفعل ال DFT لل design بتاعى واكمل باقى ال flow لحد ما طلع GDS file ... ال DFT اما بضيفه بعد ما اعمل synthesis ثم formal verification واناكد ان ال design العادى بتاعى شغال تمام ... بعد كده نحط DFT ونرجع لعمل formal verification تانى وهنا هنلاقى الى قلنا عليه انه ممكن paths كانت setup met وتبقى دلوقتى violated او ممكن انى احط DFT وانا بعمل synthesis فمن الاول هتطلع ال netlist فيها ال scan flip-flop ... هنشوف هنعمل ايه فى الحاليتين بس خلينا نمشى الخطوات من اولها

For your RTL code:

المفروض من وقت ما انت بتعمل الكود تعمل ال ports الاساسية يعنى SE SI فى ال top module و input ports و SO فى output ports of the top module وكمان ل هيبقى عند scan clk & scan reset بيقي هضيفهم ك input ports بردوا مع ال testmode كمان وساعتها هتحتاج انت تحط ال MUXs بتاعتهم لان ال tool بتشيل flip-flop وتحط ال scan flip-flop لكن مش هتحتلك MUXs لل scan clk and scan reset ... طبعا يفضل انك تعمل module لل MUX كانه block يعنى جاهز هتستخدمه زى اى block تانى وتعمله instantiation جوا ال top module بردوا فيبقى الشكل زى فى figures 21&22

```
module System_Top #(parameter WIDTH = 8, ADDR = 4, ALU_FUN_WD = 4) (
    input wire SI,
    input wire SE,
    input wire scan_clk,
    input wire scan_rst,
    input wire test_mode,
    output wire SO,
```

Figure (21) [declare the additional inputs and output due to DFT]

```
// Muxing primary clock
mux2X1 U0_mux2X1 (
    .IN_0 (CLK),
    .IN_1 (scan_clk),
    .SEL (test_mode),
    .OUT (CLK_M)
);

// Muxing resets
mux2X1 U2_mux2X1 (
    .IN_0 (RST),
    .IN_1 (scan_rst),
    .SEL (test_mode),
    .OUT (RST_M)
);
```

Figure (22) [Scan Clk and Scan RST Muxs]

متتساش انك تروح لكل ال blocks الى عاملها instantiation وفيهم clk and rst انك تغير الى مربوطين بيه فى top module وبدل
كانو CLK and RST هيقوا CLK_M and RST_M ما

Inserting DFT after synthesis:

هنتب TCL script زى ما احنا متعودين من ايام ال synthesis وبالتالي هنشوف ال commands الى بنستخدمها:

- set_scan_configuration

هنا بظبط ال configuration بتاعى او بمعنى اخر بحدد الحاجات الأساسية لل design بتاع ال scan chain بتاعى زى مثلا عدد ال
flops وايه نوع ال scan الى هستخدمه الى هو هنا استخدمنا MUX + flip-flop

Command: `set_scan_configuration -clock_mixing no_mix -style multiplexed_flip_flop -replace true`

`-max_length 100`

- **clock_mixing**

هنا بحدد هل ال blocks الى مختلفة فى clock domains يعنى كل واحد شغال ب clk مختلفة فمحطهم مع بعض فى
نفس ال chain ولا لا (هو ده المقصود بالمix) فطبعا الأحسن معملش كده واخلى ال chain كلها على
واحدة

بحيث يكون كل ال flops شغال مضبوط من غير اى اخطاء ممكن تحصل من تأثير اختلاف ال clks زى ال
Metastability

- **style**

هنا بقول شكل ال scan الى هستخدمه احنا شغالين على multiplexed_flip_flop بمعنى هيحط ال scan flip-flop
الى شرحناه وهو ده المستخدم حتى الآن

- **replace**

هنا بحدد هل بيدل ال sequential cells الى عندى الى هى ال flip-flop ب scan flip-flop الى حددت ال style
بتاعه ولا لا؟ فبقول true بحيث يبقى بيدلهم لما اجى اقوله يعمل كده ب command تانى هنشوفه بعد شوية

- **max_length**

بحدد أقصى length لل chain الى هتتخط وقولنا من ال recommendations انها تكون 100 flip-flop ...
لو احتاجت اكر ببقى هعمل كذا chain يكونوا in parallel

- set_dft_signal

يحدد DFT signal ports الى احنا ضيفناهم واوصفهم بشكل كامل بحيث يعرف ال compiler هيتعامل مع كل واحدة ازاى

Command: `set_dft_signal -type signal_type -port port_list -active state 0 OR 1 -view existing_dft`
OR spec `-usage scan OR clock_gating`

- Type

1. ScanMasterClock

ده الكلوك اللي بيشتغل وقت ال **scan operation** أثناء ال **testing**

2. MasterClock

ده الكلوك العادي بتاع ال **system functional mode**، ويستخدم أثناء مرحلة ال **capture** في ال **testing**

3. ScanClock

النوع ده بيجمع بين الاثنين (scan + capture) يعني نفس الكلوك بيقوم بالدورين. لكن لو حابب تفصل كل واحدة بكلوك مختلف، يبقى لازم تستخدم ال **ScanMasterClock** و **MasterClock** كل واحد لوحده.

4. ScanEnable

ده Signal اللي بتتحكم في ال **MUX-D Scan Flip-Flop** يعني بتحدد إذا كان الفليب فلوب هيشغل في ال **scan mode** ولا في ال **functional mode**.

5. ScanDataIn

ده ال input بتاع ال scan chain كنا مسمينه **SI** ممكن يكون في أكثر من مدخل لو السيستم الكبير متقسم لـ **sub-systems**، وكل subsystem ليه scan chain خاصة بيه.

6. ScanDataOut

ده ال output النهائي لل scan chain كنا مسمينه **SO** وبرضه ممكن يكون فيه أكثر من output لو عندنا أكثر من scan chain.

7. Reset

Signal العادية بتاعة ال **reset** أو **reset (asynchronous)** الخاصة بال design. لو عندك reset مخصص لل scan، ساعتها لازم تعرفه كـ **scan_reset** أو **scan_set**.

8. Constant

Signal ثابتة على قيمة معينة طول الوقت، متغيرش.

9. TestMode

Signal بتأخذ قيمة ثابتة لكن بتختلف حسب ال mode. يعني لو أنا في **scan mode** تبقى قيمتها ثابتة معينة، ولو في **capture mode** تبقى قيمة ثانية ثابتة مختلفة.

- Active State

بص يا سيدي، ال **active_state** دي ببساطة بتقوللي ال signal دي بتشتغل على إيه high: ولا low (يعني 1 ولا 0). ودي مهمة جدًا لل compiler عشان يعرف إزاى يتعامل معاها. وخلي بالك، مش كل الإشارات بيبقى ليها **active_state**، دي بس في شوية control signals زي **Reset, Constant, TestMode**.

- View

ال **view** معناها هل ال signal دي already موجودة في ال design ولا محتاجة أضيفها عشان ال DFT tools يتعرفوا عليها.

- لو signal أساسية زي reset أو $\text{clk} \rightarrow$ موجودة طبيعي في ال synthesis.

- لكن لو signals خاصة بال DFT زي `SE, SI, SO, testmode, scan_clk, scan_reset` → دي لازم أنا اللي أضيفها. وطبعًا في نوعين:

1. `existing_dft`: يعني ال signal دي already متوصلة وجاهزة، مش محتاج أركبها. زي مثلاً `scan_clk, scan_reset` → `testmode` دول واصلين على scan FF أو mux.
2. `spec`: هنا بقول لك tool أنا عايز أوصل signal جديدة لسه مش موجودة. وال connection ده بيتم بعدين بأوامر زي `insert_dft` ومثال: `scan enable, scan in, scan out`.

• Usage

ال `usage` يبقى بيحددلي أنا بستعمل ال signal دي في إيه. ده اللي بيخلي ال compiler يفهم يوصلها فين ويطلبها.

1. clock_gating:

لما يكون عندي gated clock في ال scan chain ، لازم أعمله bypass وقت ال `testmode` فيوصل signal زي `SE` أو `testmode` مع `gate_enable` عن طريق `OR gate`. لازم هنا أختار type صح (`ScanEnable` أو `TestMode`) وكم ال `active_state` عشان بيان تأثيرها سواء في `test mode` أو `functional mode`.

2. scan:

دي خاصة بال enable بتاع عناصر ال scan زي ال scan flip-flop. هنا النوع لازم يكون `ScanEnable`، وال usage طبعًا scan.

• Examples:-

```
set_dft_signal -port [get_ports scan_clk] -type ScanClock -view existing_dft -timing {30 60}
set_dft_signal -port [get_ports scan_rst] -type Reset -view existing_dft -active_state 0
set_dft_signal -port [get_ports test_mode] -type Constant -view existing_dft -active_state 1
set_dft_signal -port [get_ports test_mode] -type TestMode -view spec -active_state 1
set_dft_signal -port [get_ports SE] -type ScanEnable -view spec -active_state 1 -usage scan
set_dft_signal -port [get_ports clk_gate_byp] -type ScanEnable -view spec -active_state 1 -usage clock_gating
set_dft_signal -port [get_ports SI] -type ScanDataIn -view spec
set_dft_signal -port [get_ports SO] -type ScanDataOut -view spec
```

باقي الاسكريبت في الملزمه باينه غالباً مقيش حاجات ثاني عاوز اتكلم فيها لو فيه حاجه مش باينه ابعتلي بالتوفيق يرجاله